

1. `SELECT COUNT(*) FROM Student WHERE phone_number IS NULL;`
2. The student is **correct**, `DISTINCT` is redundant here. When you `GROUP BY department`, the result already has exactly one row per distinct department value (that's what grouping does — it collapses all rows sharing a department into a single output row). Since each group already produces a single, unique department value in the output, applying `DISTINCT` to a result set that already has no duplicate department values has no effect. `DISTINCT` would only matter if the `SELECT` list contained columns that weren't part of the `GROUP BY` and could repeat across groups, but here the grouped result set is already unique on department.
3. A higher row count from the `LEFT JOIN` compared to the `INNER JOIN` tells you that **some customers have no matching rows in the Order table**, i.e., there exist customers who haven't placed any orders (or whose `customer_id` doesn't match any row in Order). `INNER JOIN` only returns rows where a match exists on both sides, so customers without orders are dropped entirely. `LEFT JOIN` preserves every row from the left table (Customer) regardless of whether a match exists, filling in `NULLs` for the Order-side columns when there's no match. So the scenario causing more rows in the `LEFT JOIN` is: at least one customer exists in the Customer table who has never placed an order.
4. The query is invalid because `employee_name` appears in the `SELECT` list but is neither part of the `GROUP BY` clause nor wrapped in an aggregate function. Once you `GROUP BY department`, SQL collapses all rows for a department into a single group/row — but a department can have many employees, each with a different `employee_name`. SQL has no defined rule for picking *which* employee's name to display for that group, so standard SQL disallows it outright.
5. `SELECT department, AVG(salary)`
`FROM Employee`
`GROUP BY department`
`HAVING AVG(salary) > 80000;`
6. (a) `SELECT *`
`FROM Product`
`WHERE price > (SELECT AVG(price) FROM Product);`
 (b) `SELECT *`
`FROM Employee e1`
`WHERE salary >`
`( SELECT AVG(salary)`
`FROM Employee e2`
`WHERE e2.department = e1.department );`

Part 2:

- (a) `SELECT c.customer_id, c.name`
`FROM Customer c`
`LEFT JOIN Order o ON c.customer_id = o.customer_id`
`WHERE o.order_id IS NULL;`

```

(b) SELECT o.restaurant_id, AVG(order_totals.order_value) AS avg_order_value
FROM Order o
JOIN (
    SELECT oi.order_id, SUM(mi.price * oi.quantity) AS order_value
    FROM OrderItem oi
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY oi.order_id )
order_totals ON o.order_id = order_totals.order_id
GROUP BY o.restaurant_id
HAVING COUNT(o.order_id) > 5;

c) SELECT restaurant_id, avg_order_value
FROM (
    SELECT o.restaurant_id, AVG(order_totals.order_value) AS avg_order_value
    FROM Order o
    JOIN (
        SELECT oi.order_id, SUM(mi.price * oi.quantity) AS order_value
        FROM OrderItem oi
        JOIN MenuItem mi ON oi.item_id = mi.item_id
        GROUP BY oi.order_id
    ) order_totals ON o.order_id = order_totals.order_id
    GROUP BY o.restaurant_id
    HAVING COUNT(o.order_id) > 5
) restaurant_avgs
WHERE avg_order_value > (
    SELECT AVG(order_value) FROM (
        SELECT oi.order_id, SUM(mi.price * oi.quantity) AS order_value
        FROM OrderItem oi
        JOIN MenuItem mi ON oi.item_id = mi.item_id
        GROUP BY oi.order_id
    ) all_order_totals
);

d) SELECT mi.restaurant_id, mi.item_id, mi.item_name, item_totals.total_qty
FROM (
    SELECT mi2.restaurant_id, oi.item_id, SUM(oi.quantity) AS total_qty
    FROM OrderItem oi
    JOIN MenuItem mi2 ON oi.item_id = mi2.item_id
    GROUP BY mi2.restaurant_id, oi.item_id
) item_totals
JOIN MenuItem mi ON mi.item_id = item_totals.item_id
WHERE item_totals.total_qty = (
    SELECT MAX(it2.total_qty)
    FROM (
        SELECT mi3.restaurant_id, oi2.item_id, SUM(oi2.quantity) AS total_qty
        FROM OrderItem oi2
        JOIN MenuItem mi3 ON oi2.item_id = mi3.item_id
        GROUP BY mi3.restaurant_id, oi2.item_id
    ) it2
);

```

WHERE it2.restaurant_id = item_totals.restaurant_id
);

Part 3:

1. Functional dependencies implied by the scenario

- $\text{course_id} \rightarrow \text{course_name}, \text{instructor_id}$ (a course has one name and one instructor)
- $\text{instructor_id} \rightarrow \text{instructor_name}, \text{instructor_office}$ (an instructor has one name and one office)
- $\text{student_id} \rightarrow \text{student_name}$ (a student has one name)
- $(\text{student_id}, \text{course_id}) \rightarrow \text{grade}, \text{enrollment_date}$ (grade and enrollment date depend on the student-course pair)

2. Yes, this table is in 1NF, **assuming** each column holds a single atomic value (no repeating groups or multi-valued fields per row, e.g., a student isn't storing a list of course_ids in one cell). Given the schema as listed, each attribute appears to be atomic and there's no nested/repeating structure, so 1NF is satisfied (it just isn't in higher normal forms).

3. 2NF requires that no non-key attribute depend on only *part* of a composite primary key. Here, the primary key is $(\text{student_id}, \text{course_id})$. The partial dependencies are:

- $\text{student_id} \rightarrow \text{student_name}$ (depends only on student_id, not the full key)
- $\text{course_id} \rightarrow \text{course_name}, \text{instructor_id}, \text{instructor_name}, \text{instructor_office}$ (depends only on course_id, not the full key)

This is a problem because it creates anomalies. For example, an **update anomaly**: if a course's name changes (e.g., "Intro to DBMS" renamed to "Database Systems I"), you'd have to update course_name in *every single row* where that course appears (once per enrolled student) — if you miss even one row, the table becomes inconsistent, showing two different names for the same course.